

PHỤ LỤC B. TỰ ĐỘNG VI PHÂN (AUTODIFF)

Phụ lục này giải thích cách hoạt động của tính năng tự động vi phân (autodiff) của TensorFlow, và cách nó so sánh với các giải pháp khác.

Giả sử bạn định nghĩa một hàm $f(x, y) = x^2y + y + 2$, và bạn cần các đạo hàm riêng $\partial f / \partial x$ và $\partial f / \partial y$, thường là để thực hiện tối ưu hóa gradient descent (hoặc một thuật toán tối ưu hóa khác). Các lựa chọn chính của bạn là vi phân thủ công, xấp xỉ sai phân hữu hạn, autodiff chế độ xuôi (forward-mode autodiff), và autodiff chế độ ngược (reverse-mode autodiff). TensorFlow triển khai autodiff chế độ ngược, nhưng để hiểu nó, hữu ích khi xem xét các lựa chọn khác trước. Vì vậy, hãy đi qua từng lựa chọn, bắt đầu với vi phân thủ công.

Vi Phân Thủ Công (Manual Differentiation)

Cách tiếp cận đầu tiên để tính đạo hàm là cầm bút chì và một tờ giấy và sử dụng kiến thức giải tích của bạn để tìm ra phương trình thích hợp. Đối với hàm $f(x, y)$ vừa định nghĩa, không quá khó; bạn chỉ cần sử dụng năm quy tắc:

- Đạo hàm của một hằng số là 0.
- Đạo hàm của λx là λ (trong đó λ là một hằng số).
- Đạo hàm của x^λ là $\lambda x^{\lambda-1}$, vì vậy đạo hàm của x^2 là $2x$.
- Đạo hàm của tổng các hàm là tổng các đạo hàm của các hàm đó.
- Đạo hàm của λ nhân với một hàm là λ nhân với đạo hàm của nó.

Từ các quy tắc này, bạn có thể tìm ra Phương trình B-1.

Phương trình B-1. Các đạo hàm riêng của $f(x, y)$

$$\frac{\partial f}{\partial x} = \frac{\partial(x^2y)}{\partial x} + \frac{\partial y}{\partial x} + \frac{\partial 2}{\partial x} = y \frac{\partial x^2}{\partial x} + 0 + 0 = y(2x) = 2xy$$
$$\frac{\partial f}{\partial y} = \frac{\partial(x^2y)}{\partial y} + \frac{\partial y}{\partial y} + \frac{\partial 2}{\partial y} = x^2 \frac{\partial y}{\partial y} + 1 + 0 = x^2(1) + 1 = x^2 + 1$$

Cách tiếp cận này có thể trở nên rất tẻ nhạt đối với các hàm phức tạp hơn, và bạn có nguy cơ mắc lỗi. May mắn thay, có những lựa chọn khác. Bây giờ chúng ta hãy xem xét xấp xỉ sai phân hữu hạn.

Xấp Xỉ Sai Phân Hữu Hạn (Finite Difference Approximation)

Nhắc lại rằng đạo hàm $h'(x_0)$ của một hàm $h(x)$ tại một điểm x_0 là độ dốc của hàm tại điểm đó. Chính xác hơn, đạo hàm được định nghĩa là giới hạn của độ dốc của một đường thẳng đi qua điểm x_0 này và một điểm x khác trên hàm, khi x tiến gần vô hạn đến x_0 (xem Phương trình B-2).

Phương trình B-2. Định nghĩa đạo hàm của một hàm $h(x)$ tại điểm x_0

$$h'(x_0) = \lim_{x \rightarrow x_0} \frac{h(x) - h(x_0)}{x - x_0} = \lim_{\epsilon \rightarrow 0} \frac{h(x_0 + \epsilon) - h(x_0)}{\epsilon}$$

Vì vậy, nếu chúng ta muốn tính đạo hàm riêng của $f(x, y)$ đối với x tại $x = 3$ và $y = 4$, chúng ta có thể tính $f(3 + \text{epsilon}, 4) - f(3, 4)$ và chia kết quả cho epsilon , sử dụng một giá trị rất nhỏ cho epsilon . Loại xấp xỉ số học này của đạo hàm được gọi là **xấp xỉ sai phân hữu hạn (finite difference approximation)**, và phương trình cụ thể này được gọi là **thương số sai phân của Newton (Newton's difference quotient)**. Đó chính xác là những gì mã sau đây thực hiện:

```
def f(x, y):
    return x**2 * y + y + 2

def derivative(f, x, y, x_eps, y_eps):
    # Đây là một hàm helper để minh họa ý tưởng, không phải cách tính đạo hàm chính xác
    # Nếu x_eps và y_eps đều khác 0, nó tính đạo hàm theo hướng chéo, không phải đạo hàm riêng
    # Để tính đạo hàm riêng, một trong x_eps hoặc y_eps phải là 0
    return (f(x + x_eps, y + y_eps) - f(x, y)) / (x_eps + y_eps)

# Để tính đạo hàm riêng theo x:
# x_eps = một số rất nhỏ, y_eps = 0
df_dx = (f(3 + 0.00001, 4) - f(3, 4)) / 0.00001
# Để tính đạo hàm riêng theo y:
# x_eps = 0, y_eps = một số rất nhỏ
df_dy = (f(3, 4 + 0.00001) - f(3, 4)) / 0.00001
```

Thật không may, kết quả không chính xác (và nó trở nên tệ hơn đối với các hàm phức tạp hơn). Các kết quả chính xác lần lượt là 24 và 10, nhưng thay vào đó chúng ta nhận được:

```
>>> df_dx 24.000039999805264
>>> df_dy 10.000000000331966
```

Lưu ý rằng để tính cả hai đạo hàm riêng, chúng ta phải gọi hàm $f()$ ít nhất ba lần (chúng ta đã gọi nó bốn lần trong mã trước, nhưng có thể tối ưu hóa). Nếu có 1.000 tham số, chúng ta sẽ cần gọi $f()$ ít nhất 1.001 lần. Khi bạn đang xử lý các mạng nơ-ron lớn, điều này làm cho xấp xỉ sai phân hữu hạn trở nên quá không hiệu quả.

Tuy nhiên, phương pháp này rất đơn giản để triển khai nên nó là một công cụ tuyệt vời để kiểm tra xem các phương pháp khác có được triển khai đúng cách hay không. Ví dụ, nếu nó không khớp với hàm bạn đã đạo hàm thủ công, thì hàm của bạn có thể chứa lỗi.

Cho đến nay, chúng ta đã xem xét hai cách để tính gradient: sử dụng vi phân thủ công và sử dụng xấp xỉ sai phân hữu hạn. Thật không may, cả hai đều có những hạn chế nghiêm trọng khi huấn luyện một mạng nơ-ron quy mô lớn. Vì vậy, hãy chuyển sang autodiff, bắt đầu với chế độ xuôi.

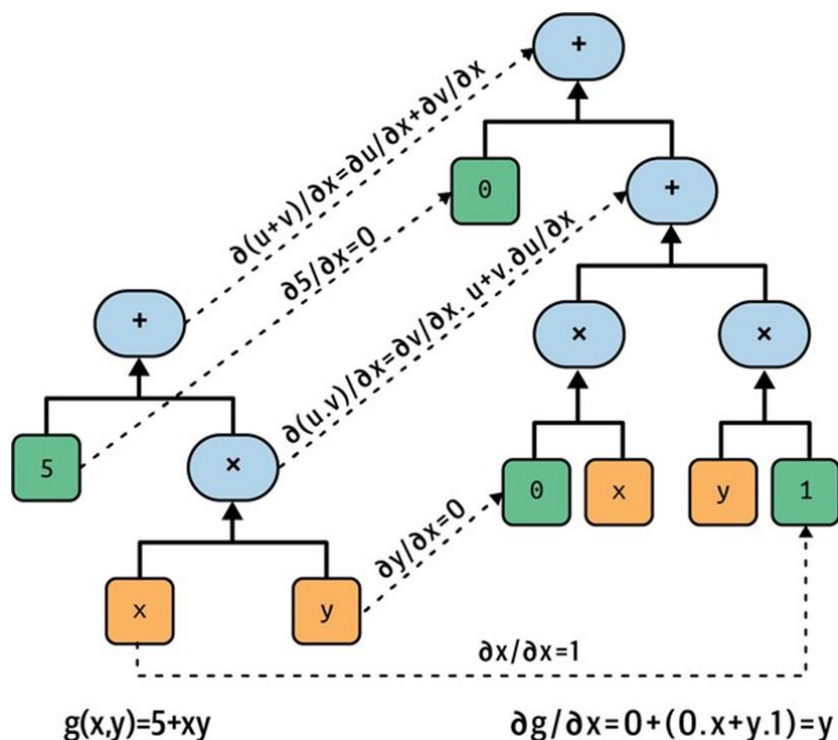
Autodiff Chế Độ Xuôi (Forward-Mode Autodiff)

Hình B-1 cho thấy cách hoạt động của autodiff chế độ xuôi trên một hàm thậm chí còn đơn giản hơn, $g(x, y) = 5 + xy$. Đồ thị cho hàm đó được biểu diễn ở bên trái. Sau autodiff chế độ xuôi, chúng ta có đồ thị ở bên phải, biểu diễn đạo hàm riêng $\partial g / \partial x = 0 + (0 \times x + y \times 1) = y$ (chúng ta cũng có thể tìm đạo hàm riêng đối với y một cách tương tự).

Thuật toán sẽ đi qua đồ thị tính toán từ đầu vào đến đầu ra (do đó có tên “chế độ xuôi”). Nó bắt đầu bằng cách lấy các đạo hàm riêng của các nút lá. Nút hằng số (5) trả về hằng số 0, vì đạo hàm của một hằng số luôn là 0. Biến x trả về hằng số 1 vì $\partial x / \partial x = 1$, và biến y trả về hằng số 0 vì $\partial y / \partial x = 0$ (nếu chúng ta đang tìm đạo hàm riêng đối với y , nó sẽ ngược lại).

Bây giờ chúng ta có tất cả những gì cần thiết để di chuyển lên đồ thị đến nút nhân trong hàm g . Giải tích cho chúng ta biết rằng đạo hàm của tích của hai hàm u và v là $\partial(u \times v) / \partial x = \partial u / \partial x \times v + u \times \partial v / \partial x$. Do đó, chúng ta có thể xây dựng một phần lớn của đồ thị ở bên phải, biểu diễn $0 \times x + y \times 1$.

Cuối cùng, chúng ta có thể đi lên nút cộng trong hàm g . Như đã đề cập, đạo hàm của tổng các hàm là tổng các đạo hàm của các hàm đó, vì vậy chúng ta chỉ cần tạo một nút cộng và kết nối nó với các phần của đồ thị mà chúng ta đã tính toán. Chúng ta nhận được đạo hàm riêng chính xác: $\partial g / \partial x = 0 + (0 \times x + y \times 1) = y$.



Hình B-1. Autodiff chế độ xuôi.

Tuy nhiên, phương trình này có thể được đơn giản hóa (rất nhiều). Bằng cách áp dụng một vài bước cắt tĩa đồ thị tính toán để loại bỏ tất cả các phép toán không cần thiết, chúng ta có được một đồ thị nhỏ hơn nhiều chỉ với một nút: $\text{partial}g/\text{partial}x = y$. Trong trường hợp này, việc đơn giản hóa khá dễ dàng, nhưng đối với một hàm phức tạp hơn, autodiff chế độ xuôi có thể tạo ra một đồ thị khổng lồ mà có thể khó đơn giản hóa và dẫn đến hiệu suất không tối ưu.

Lưu ý rằng chúng ta bắt đầu với một đồ thị tính toán, và autodiff chế độ xuôi tạo ra một đồ thị tính toán khác. Đây được gọi là **vi phân ký hiệu (symbolic differentiation)**, và nó có hai tính năng hay: thứ nhất, một khi đồ thị tính toán của đạo hàm đã được tạo ra, chúng ta có thể sử dụng nó bao nhiêu lần tùy thích để tính đạo hàm của hàm đã cho với bất kỳ giá trị nào của x và y ; thứ hai, chúng ta có thể chạy lại autodiff chế độ xuôi trên đồ thị kết quả để nhận được đạo hàm bậc hai nếu chúng ta cần (tức là đạo hàm của đạo hàm). Chúng ta thậm chí có thể tính đạo hàm bậc ba, v.v.

Nhưng cũng có thể chạy autodiff chế độ xuôi mà không cần xây dựng đồ thị (tức là theo số học, không phải ký hiệu), chỉ bằng cách tính toán các kết quả trung gian ngay lập tức. Một cách để làm điều này là sử dụng **số kép (dual numbers)**, là những số kỳ lạ nhưng hấp dẫn có dạng $a + b\epsilon$, trong đó a và b là số thực và ϵ là một số vô cùng nhỏ sao cho $\epsilon^2 = 0$ (nhưng $\epsilon \neq 0$). Bạn có thể nghĩ số kép $42 + 24\epsilon$ như một cái gì đó tương tự $42.0000\dots000024$ với vô số số 0 (nhưng tất nhiên đây chỉ là đơn giản hóa để bạn có ý tưởng về số kép). Một số kép được biểu diễn trong bộ nhớ dưới dạng một cặp số float. Ví dụ, $42 + 24\epsilon$ được biểu diễn bởi cặp $(42.0, 24.0)$.

Số kép có thể được cộng, nhân, v.v., như được thể hiện trong Phương trình B-3.

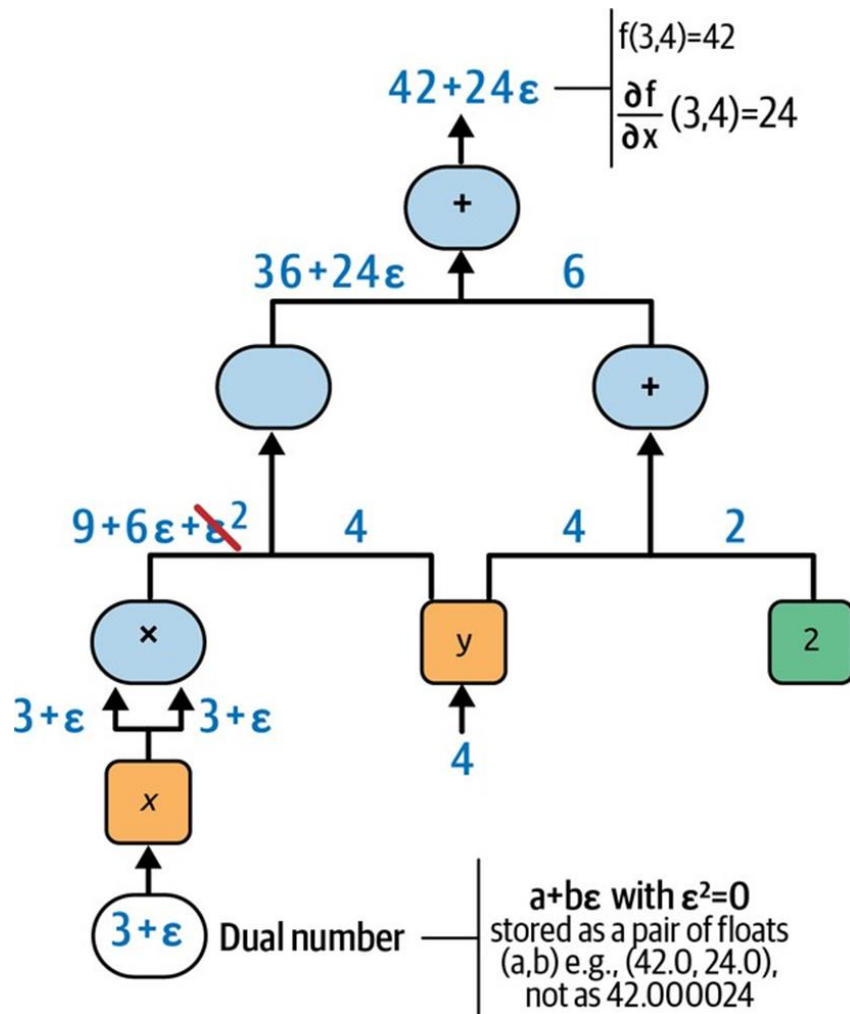
Phương trình B-3. Một vài phép toán với số kép

Công thức B-3: Một vài phép toán với số kép

- $\lambda(a + b\epsilon) = \lambda a + \lambda b\epsilon$
- $(a + b\epsilon) + (c + d\epsilon) = (a + c) + (b + d)\epsilon$
- $(a + b\epsilon) \times (c + d\epsilon) = ac + (ad + bc)\epsilon + (bd)\epsilon^2 = ac + (ad + bc)\epsilon$

Quan trọng nhất, có thể chứng minh rằng $h(a + b\epsilon) = h(a) + bh'(a)\epsilon$. Do đó, việc tính $h(a + \epsilon)$ sẽ cho bạn cả $h(a)$ và đạo hàm $h'(a)$ chỉ trong một lần tính.

Điều này cho phép tính đạo hàm riêng của một hàm. Ví dụ, để tính đạo hàm riêng của $f(x, y)$ theo x tại $x = 3$ và $y = 4$ (ký hiệu là $\partial f / \partial x(3, 4)$), chúng ta có thể sử dụng số kép. Tất cả những gì chúng ta cần làm là tính $f(3 + \epsilon, 4)$; kết quả sẽ là một số kép có thành phần đầu tiên bằng với $f(3, 4)$ và thành phần thứ hai bằng với $\partial f / \partial x(3, 4)$.



Hình B-2. Autodiff chế độ xuôi sử dụng số kép.

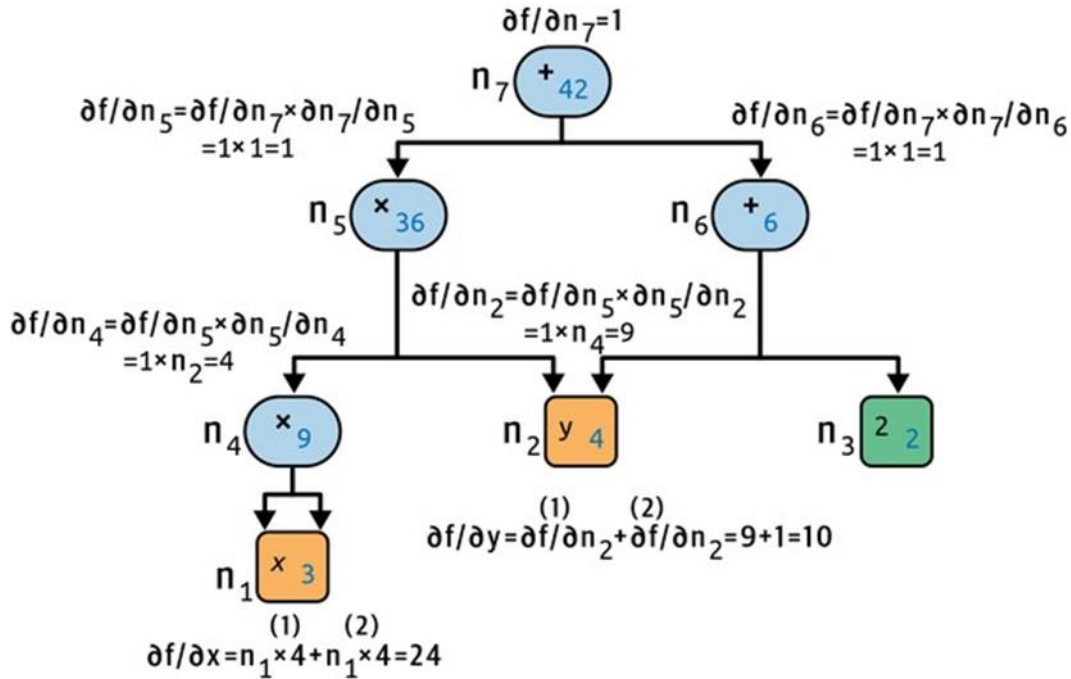
Để tính $\text{partial}f/\text{partial}y(3,4)$, chúng ta sẽ phải đi qua đồ thị một lần nữa, nhưng lần này với $x = 3$ và $y = 4 + \text{epsilon}$.

Vì vậy, autodiff chế độ xuôi chính xác hơn nhiều so với xấp xỉ sai phân hữu hạn, nhưng nó mắc phải cùng một lỗi lớn, ít nhất là khi có nhiều đầu vào và ít đầu ra (như trường hợp xử lý mạng nơ-ron): nếu có 1.000 tham số, nó sẽ yêu cầu 1.000 lần đi qua đồ thị để tính tất cả các đạo hàm riêng. Đây là nơi autodiff chế độ ngược phát huy tác dụng: nó có thể tính tất cả chúng chỉ trong hai lần đi qua đồ thị. Hãy xem cách thực hiện.

Autodiff Chế Độ Ngược (Reverse-Mode Autodiff)

Autodiff chế độ ngược là giải pháp được TensorFlow triển khai. Nó đầu tiên đi qua đồ thị theo hướng xuôi (tức là từ đầu vào đến đầu ra) để tính giá trị của mỗi nút. Sau đó, nó thực hiện một lần đi qua thứ hai, lần này theo hướng ngược lại (tức là từ đầu ra đến đầu vào), để tính tất cả các đạo hàm riêng. Tên “chế độ ngược” xuất phát từ lần đi qua thứ hai này qua đồ thị, nơi các gradient chảy theo hướng ngược lại. Hình B-3 biểu diễn lần đi qua thứ hai. Trong lần đi qua đầu tiên, tất cả các giá trị nút đã được tính toán, bắt đầu từ $x = 3$ và $y = 4$. Bạn có

thể thấy các giá trị đó ở phía dưới bên phải của mỗi nút (ví dụ: $x \text{ times } x = 9$). Các nút được dán nhãn n_1 đến n_7 để rõ ràng. Nút đầu ra là $n_7 : f(3,4) = n_7 = 42$.



Hình B-3. Autodiff chế độ ngược.

Ý tưởng là dần dần đi xuống đồ thị, tính đạo hàm riêng của $f(x, y)$ đối với mỗi nút liên tiếp, cho đến khi chúng ta đạt đến các nút biến. Đối với điều này, autodiff chế độ ngược dựa rất nhiều vào **quy tắc chuỗi (chain rule)**, được thể hiện trong Phương trình B-4.

Công thức B-4: Quy tắc chuỗi

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial n_i} \times \frac{\partial n_i}{\partial x}$$

Vì n_7 là nút đầu ra, nên $f = n_7$, do đó $\frac{\partial f}{\partial n_7} = 1$.

Hãy tiếp tục đi xuống đồ thị: f thay đổi như thế nào khi n_5 thay đổi? Câu trả lời là $\frac{\partial f}{\partial n_5} = \frac{\partial f}{\partial n_7} \times \frac{\partial n_7}{\partial n_5}$. Chúng ta đã biết $\frac{\partial f}{\partial n_7} = 1$. Vì n_7 chỉ đơn giản là thực hiện phép tổng $n_5 + n_6$, nên ta thấy $\frac{\partial n_7}{\partial n_5} = 1$. Do đó, $\frac{\partial f}{\partial n_5} = 1 \times 1 = 1$.

Bây giờ chúng ta có thể tiếp tục đến nút n_4 : f thay đổi bao nhiêu khi n_4 thay đổi? Câu trả lời là $\frac{\partial f}{\partial n_4} = \frac{\partial f}{\partial n_5} \times \frac{\partial n_5}{\partial n_4}$. Vì $n_5 = n_4 \times n_2$, nên ta thấy $\frac{\partial n_5}{\partial n_4} = n_2$. Do đó, $\frac{\partial f}{\partial n_4} = 1 \times n_2 = 4$.

Quá trình này tiếp tục cho đến khi chúng ta đi đến đáy của đồ thị. Tại thời điểm đó, chúng ta đã tính tất cả các đạo hàm riêng của $f(x, y)$ tại điểm $x = 3$ và $y = 4$. Trong ví dụ này, chúng ta tìm thấy $\frac{\partial f}{\partial x} = 24$ và $\frac{\partial f}{\partial y} = 10$. Nghe có vẻ đúng!

Quá trình này tiếp tục cho đến khi chúng ta đạt đến đáy của đồ thị. Tại thời điểm đó, chúng ta sẽ đã tính toán tất cả các đạo hàm riêng của $f(x, y)$ tại điểm $x = 3$ và $y = 4$. Trong ví dụ này, chúng ta tìm thấy $\text{partial}f/\text{partial}x = 24$ và $\text{partial}f/\text{partial}y = 10$. Nghe có vẻ đúng!

Autodiff chế độ ngược là một kỹ thuật rất mạnh mẽ và chính xác, đặc biệt khi có nhiều đầu vào và ít đầu ra, vì nó chỉ yêu cầu một lần đi qua xuôi cộng với một lần đi qua ngược cho mỗi đầu ra để tính tất cả các đạo hàm riêng cho tất cả các đầu ra đối với tất cả các đầu vào. Khi huấn luyện mạng nơ-ron, chúng ta thường muốn tối thiểu hóa hàm mất mát (loss), vì vậy chỉ có một đầu ra (hàm mất mát), và do đó chỉ cần hai lần đi qua đồ thị để tính gradient. Autodiff chế độ ngược cũng có thể xử lý các hàm không hoàn toàn khả vi, miễn là bạn yêu cầu nó tính đạo hàm riêng tại các điểm khả vi.

Trong Hình B-3, các kết quả số học được tính toán ngay lập tức, tại mỗi nút. Tuy nhiên, đó không phải chính xác những gì TensorFlow làm: thay vào đó, nó tạo ra một đồ thị tính toán mới. Nói cách khác, nó triển khai **autodiff chế độ ngược ký hiệu (symbolic reverse-mode autodiff)**. Bằng cách này, đồ thị tính toán để tính gradient của hàm mất mát đối với tất cả các tham số trong mạng nơ-ron chỉ cần được tạo một lần, và sau đó nó có thể được thực thi lặp đi lặp lại, bất cứ khi nào bộ tối ưu hóa cần tính gradient. Hơn nữa, điều này giúp có thể tính toán các đạo hàm bậc cao hơn nếu cần.